

# CryptoChain Analyzer Dashboard

## Final report - Cryptography (UAX, 2025-26)

Author: **Gaizka** | GitHub: **Gaizka8** | Repository: [blockchain-dashboard-Gaizka8-1](#)

## 1. Cryptographic metrics displayed

The dashboard surfaces four families of metrics, all derived directly from the live Bitcoin chain via the Blockstream REST API. Each one is a direct application of Section 6 of the Blockchain notes (SHA-256, Proof of Work, difficulty target).

| Metric              | Where it is computed                                                                              | What it means                                                                                                                                                                                           |
|---------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Difficulty          | M1, M3 (decoded from <i>bits</i> via $\text{target} = \text{mantissa} \cdot 256^{\text{exp}-3}$ ) | Ratio $\text{MAX\_TARGET} / \text{target}$ . A difficulty of $1.3 \times 10^{14}$ means the network must, on average, perform $\sim 1.3 \times 10^{14} \times 2^{32}$ hashes to produce a single block. |
| Target threshold    | M1 (visualised as a $\log_2$ bar inside the $2^{256}$ space)                                      | The numerical ceiling that the double-SHA256 of a valid header must stay below. Equivalent to requiring N leading zero bits.                                                                            |
| Inter-block times   | M1 (histogram), M4 (statistical model)                                                            | Empirical distribution of seconds between consecutive blocks. Should follow $\text{Exp}(1/600)$ under the Poisson model.                                                                                |
| Network hash rate   | M1 ( $\text{difficulty} \times 2^{32} / \text{mean}(\Delta t)$ )                                  | Estimate of the global hashing power, in EH/s for current Bitcoin.                                                                                                                                      |
| Local verification  | PoW M2 (hashlib double SHA-256 of the 80-byte header)                                             | Re-derives the block hash from the raw header and checks $\text{hash} < \text{target}$ . Confirms the cryptographic chain rather than trusting the explorer.                                            |
| Difficulty retarget | M3 (one sample every 2016 blocks)                                                                 | Plots difficulty over the last K epochs and the ratio $\text{actual} / \text{target}$ period time, illustrating the self-correcting feedback loop.                                                      |

**Worked example.** At the time of writing, block #948 592 had *bits* = 0x17030c63 and difficulty  $\approx 1.325 \times 10^{14}$ . Running `SHA256(SHA256(header))` in Python (M2) produced exactly the hash that Blockstream reports (000000000000000000000000154eb...), with 79 leading zero bits. Multiplied by  $2^{32}$  and divided by the observed mean inter-block time, this gives a network hash-rate estimate in the EH/s range, consistent with public mining-pool figures.

## 2. AI component (M4)

**Problem.** Bitcoin block times follow a Poisson process, so consecutive inter-arrival times should be i.i.d.  $\text{Exp}(1/600)$ . Long stalls or unusually fast bursts can correlate with mining-pool behaviour, network splits, or measurement issues; we want to flag them automatically.

**Model chosen.** A two-sided exponential-tail test. After estimating the rate by maximum likelihood ( $\lambda = 1 / \text{sample mean}$ ), the per-block p-value is  $p = 2 \cdot \min(F(t), 1 - F(t))$  with  $F(t) = 1 - \exp(-\lambda t)$ . A block is flagged when  $p < \alpha$  (default  $\alpha = 0.01$ ). I picked this model over heavier alternatives (LSTM, Prophet) because the underlying generative process is known and one-dimensional, so a parametric test is both more interpretable *and* more data-efficient. An **Isolation Forest** from scikit-learn is kept as a non-parametric baseline.

**Training data.** The last 200 inter-block times pulled live from [blockstream.info](https://blockstream.info). The detectors are unsupervised, so the same window is used for fitting and scoring (no leakage of future blocks).

**Evaluation.** Because mining-pool ground truth is not publicly labelled, the dashboard injects synthetic anomalies into the real series (5% of the samples replaced by very long stalls or very short bursts) and reports Precision, Recall, F1 and ROC-AUC against those known labels. Representative numbers from a

200-sample run with 10 injected anomalies:

| Detector                               | Precision | Recall | F1   | ROC-AUC |
|----------------------------------------|-----------|--------|------|---------|
| Statistical (Exp tail, $\alpha=0.01$ ) | 0.71      | 0.50   | 0.59 | 0.97    |
| Isolation Forest (5% contamination)    | 0.50      | 0.50   | 0.50 | 0.87    |

**Reading the numbers.** The statistical detector dominates on ROC-AUC because it is correctly specified for the data-generating process; its Precision is high (few false positives) but its Recall at  $\alpha = 0.01$  is moderate, which is the price paid for being conservative. The Isolation Forest catches similar magnitudes but with no notion of significance, so picking the *contamination* hyper-parameter requires guessing the anomaly rate. **Limitations:** the model assumes stationarity within the window, ignores the small bias introduced by miners' timestamp manipulation, and the synthetic evaluation is an upper bound on real-world performance.

### 3. References

- [1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Section 4 (Proof-of-Work) and Section 11 (Calculations).
- [2] Antonopoulos, A. M. (2017). *Mastering Bitcoin*, 2nd edition, O'Reilly. Chapter 10, 'Mining and Consensus'.
- [3] Blockstream. *Esplora HTTP REST API*. [github.com/Blockstream/esplora/blob/master/API.md](https://github.com/Blockstream/esplora/blob/master/API.md)
- [4] Bowden, R. et al. (2018). *Block arrivals in the Bitcoin blockchain*. arXiv:1801.07447 - empirical evidence that the exponential-Poisson assumption is a good first-order model with measurable deviations.